

Aggregation, grain and business measures

Many rows become one number.

BY THE END OF TODAY, YOU CAN

Class objectives

-
- 01 State a grouped result's grain before running
 - 02 Explain why three counts differ
 - 03 Filter rows with WHERE and groups with HAVING
 - 04 Build a column from a rule with CASE
 - 05 Define a KPI, then write its query
-

Warm-up

Five questions on yesterday, ungraded. The last one asks which part felt hardest.

Suggested time • 10 minutes

Rows collapse into groups

Twelve rows of D03.order_items, grouped two ways.

A new column on every row

One row of order_items is one product on one order. Twelve rows.

Suggested time • 8 minutes

[labs/D04/01-Ins-Collapse-Rows/README.md](#)

A calculation adds a column

qty * price on every row. Still 12 rows: the row count did not change.

```
SELECT item_id, order_id, product_id, qty, price,  
       qty * price AS line_revenue  
FROM D03.order_items  
ORDER BY item_id;
```

→ **12 rows**

GROUP BY collapses rows

Before you run: how many rows come back, and what is one row?

Suggested time • 10 minutes

[labs/D04/01-Ins-Collapse-Rows/README.md](#)

DEFINITION

GROUP BY collapses the rows that share a value into one row each, so the grain of the result is whatever you group by.

GROUP BY order_id

12 rows become 5 groups

Order 1001: 2 rows become 1 row. One row of the result is one order.

item_id	order_id	qty	price
9001	1001	2	19.50
9002	1001	1	8.00
9003	1002	3	19.50
9004	1002	1	42.00
9005	1003	2	8.00
9006	1003	1	26.00
9007	1003	1	19.50
9008	1004	2	42.00
9009	1004	3	26.00
9010	1005	4	19.50
9011	1005	1	8.00
9012	1005	2	15.00



order_id	lines	order_revenue
1001	2	47.00
1002	2	100.50
1003	3	61.50
1004	2	162.00
1005	3	116.00

GROUP BY order_id

12 rows become 5 groups

Order 1003: 3 rows become 1 row, with 3 lines and 61.50.

item_id	order_id	qty	price
9001	1001	2	19.50
9002	1001	1	8.00
9003	1002	3	19.50
9004	1002	1	42.00
9005	1003	2	8.00
9006	1003	1	26.00
9007	1003	1	19.50
9008	1004	2	42.00
9009	1004	3	26.00
9010	1005	4	19.50
9011	1005	1	8.00
9012	1005	2	15.00



order_id	lines	order_revenue
1001	2	47.00
1002	2	100.50
1003	3	61.50
1004	2	162.00
1005	3	116.00

GROUP BY sets the grain

One row per **order_id**. COUNT and SUM work inside each group.

```
SELECT order_id,  
       COUNT(*) AS lines,  
       SUM(qty * price) AS order_revenue  
FROM D03.order_items  
GROUP BY order_id  
ORDER BY order_id;
```

→ 5 rows

Every column: grouped or summarised

The error: product_id is neither grouped nor aggregated.

```
SELECT order_id, product_id,  
        SUM(qty * price) AS order_revenue  
FROM D03.order_items  
GROUP BY order_id;
```

Order 1003 has three products. Which one goes in its row?

The aggregate functions, checked by hand

Predict each number from the D01 worksheet or the rows on screen, then run.

Suggested time • 12 minutes

[labs/D04/02-Evr-Aggregate-Functions/README.md](https://github.com/DataCamp/labs/blob/main/D04/02-Evr-Aggregate-Functions/README.md)

Seven functions on the D03 tables

COUNT(city) skips NULL: 4 of the 5 customers have a city.

Query	Result
COUNT(*) FROM D03.orders	5
COUNT(city) FROM D03.customers	4
COUNT(DISTINCT customer_id) FROM D03.orders	3
SUM(qty * price) FROM D03.order_items	487.00
ROUND(AVG(price), 2) FROM D03.order_items	21.08
MIN(price), MAX(price) FROM D03.order_items	8.00 · 42.00
MIN(order_date), MAX(order_date) FROM D03.orders	2026-08-03 · 2026-08-24

Nakheel Retail: counts, sums and filters on groups

The business you work with from now until November.

Upload Nakheel Retail

Dataset nakheel, Multi-region US. Checkpoint: 500, 60, 3,000 and 7,516 rows.

Suggested time • 10 minutes

[labs/D04/03-Evr-Upload-Nakheel/README.md](#)

Four tables, 24 months

Clothing and footwear in eight Jordanian cities, in stores and online, September 2024 to August 2026.

Table	One row is one	Rows
nakheel.customers	customer record	500
nakheel.products	product	60
nakheel.orders	order	3,000
nakheel.order_items	product on one order	7,516

ONE ROW IS ONE ORDER

nakheel.orders

The first 5 of 3,000 rows. Six of its nine columns.

Primary key

Foreign key

order_id	customer_id	order_date	channel	status	order_total
100001	C-1267	2024-09-01	store	completed	22.50
100002	C-1348	2024-09-02	online	completed	43.50
100003	C-1357	2024-09-03	store	completed	47.00
100004	C-1335	2024-09-03	store	completed	120.00
100005	C-1292	2024-09-03	store	completed	106.50

Group and count Nakheel orders

Orders by status, revenue by month, then three counts on the same rows.

Suggested time • 10 minutes

[labs/D04/04-Evr-Count-Three-Ways/README.md](#)

COUNT(column) skips NULL

2,771 of 3,000 orders have a delivery date. Cancelled and pending orders have none.

```
SELECT COUNT(*) AS orders,  
       COUNT(delivered_date) AS delivered_orders,  
       COUNT(DISTINCT customer_id) AS customers_who_ordered  
FROM nakheel.orders;
```

→ 3,000 • 2,771 • 462

COUNT(DISTINCT) counts different values

462 different customers placed the 3,000 orders.

```
SELECT COUNT(*) AS orders,  
       COUNT(delivered_date) AS delivered_orders,  
       COUNT(DISTINCT customer_id) AS customers_who_ordered  
FROM nakheel.orders;
```

→ 3,000 • 2,771 • 462

WHERE filters rows, HAVING filters groups

Two filters, and two new steps on the run order.

Suggested time • 5 minutes

[labs/D04/05-Ins-Where-vs-Having/README.md](#)

HAVING keeps groups that pass

WHERE drops rows first; **HAVING** drops groups after counting.

```
SELECT customer_id, COUNT(*) AS completed_orders
FROM nakheel.orders
WHERE status = 'completed'
GROUP BY customer_id
HAVING COUNT(*) > 20
ORDER BY completed_orders DESC, customer_id;
```

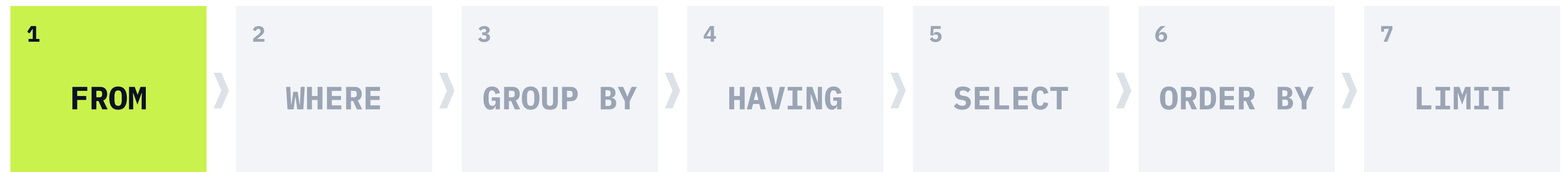
→ **22 rows**

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



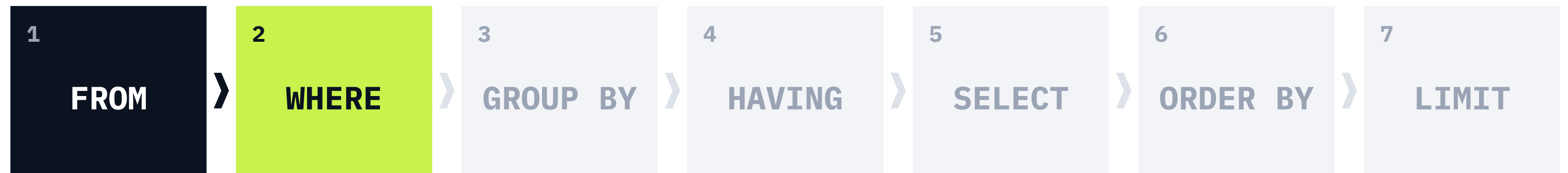
FROM runs first: which table.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



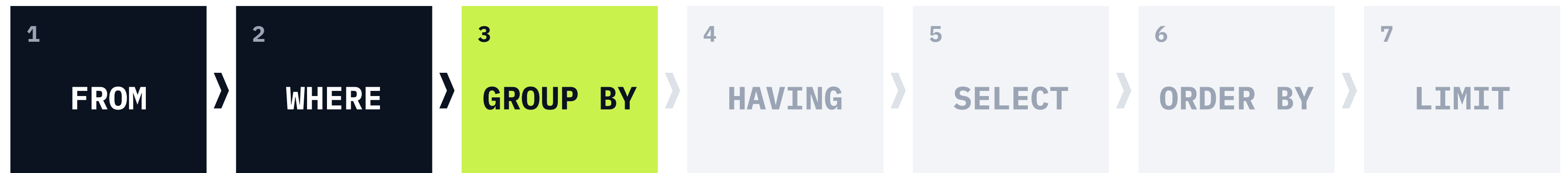
WHERE keeps rows before any counting, so COUNT(*) is not allowed there.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



GROUP BY collapses the rows that share a value into one row each.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



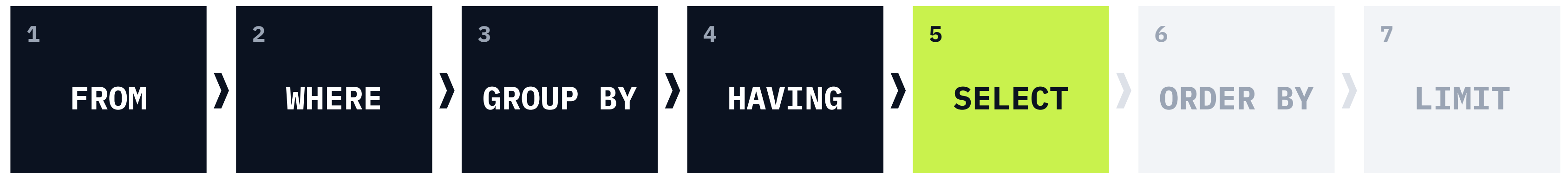
HAVING keeps the groups whose count or sum passes the test.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



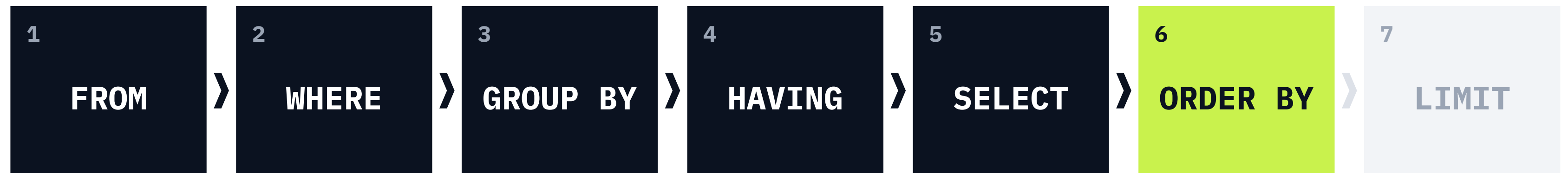
SELECT picks the columns and names them.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



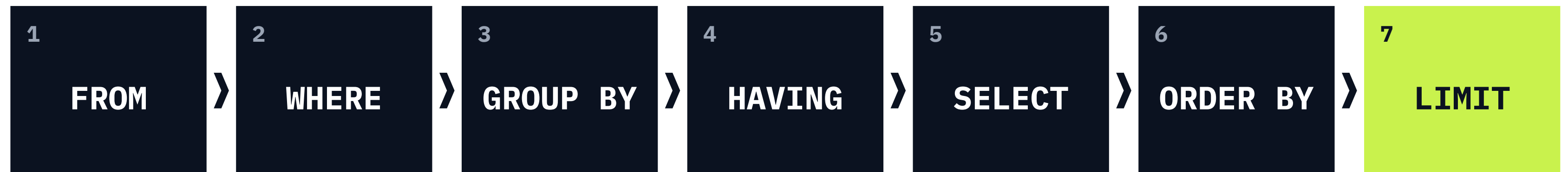
ORDER BY sorts the result.

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



LIMIT runs last: how many rows to keep.

STUDENT DO

Activity: First measures on Nakheel

In this activity, you will answer three questions from Nakheel's managers with GROUP BY, WHERE and HAVING.

Suggested time

10 minutes

[labs/D04/06-Stu-Nakheel-Measures/README.md](#)

Activity: First measures on Nakheel

INSTRUCTIONS

1. Completed orders and their value, by channel.
 2. Months of 2025 with more than 13,000 dinars of completed orders.
 3. Customers in each city, biggest city first.
- Before each query: what is one row of the result?

HINT

Question 2 filters rows and groups: one test in WHERE, one in HAVING.

BONUS

Which single day had the most orders?

Time's up. Let's review.

First measures on Nakheel

Solved: `labs/D04/06-Stu-Nakheel-Measures/Solved/nakheel_measures.sql`

Make new columns from rules

Write the rule in words first, then the expression.

Write the rule, then the CASE

Large from 200 dinars, medium from 50, otherwise small.

Suggested time • 8 minutes

[labs/D04/07-Ins-Case-Rules/README.md](#)

CASE turns a rule into a column

CASE stops at the first true WHEN.

```
SELECT order_id, order_total,  
       CASE  
         WHEN order_total >= 200 THEN 'large'  
         WHEN order_total >= 50 THEN 'medium'  
         ELSE 'small'  
       END AS size_band  
FROM nakheel.orders;
```

GROUP BY THE NEW COLUMN

Completed orders by size band

Large orders are 11% of completed orders and about half of completed revenue.

size_band	completed_orders	revenue
large	317	155,692.50
medium	1,342	128,783.00
small	1,112	31,548.00

Dates

Group Nakheel orders by year, by month and by weekday.

Suggested time • 10 minutes

[labs/D04/08-Evr-Dates/README.md](#)

Three ways to group by date

2024 has four months of data and 2026 has eight. That returns in the lab.

Expression	Groups	One value
EXTRACT(YEAR FROM order_date)	3	2025: 1,442 orders
DATE_TRUNC(order_date, MONTH)	24	2024-09-01: 123 orders
FORMAT_DATE('%A', order_date)	7	Friday lowest: 357

Clean text in one line

The D01 spelling problem, fixed with TRIM and LOWER.

Suggested time • 7 minutes

[labs/D04/09-Ins-Clean-Text/README.md](#)

Fix the spelling in one line

TRIM removes end spaces; **LOWER** makes letters small.

```
SELECT LOWER(TRIM(segment)) AS segment_clean,  
       COUNT(*) AS customers  
FROM nakheel.customers  
GROUP BY segment_clean  
ORDER BY customers DESC;
```

→ 2 rows: retail 435, wholesale 65

Conditional counts and rates

Count the cancelled orders without filtering the others away. Checkpoint: 0.0707.

Suggested time • 10 minutes

[labs/D04/10-Evr-Rates/README.md](#)

The average of a 0/1 column is a rate

Four example rows.

status	is_cancelled
completed	0
completed	0
cancelled	1
completed	0



$$\text{AVG} = 0.25$$

One order in four was cancelled.

Cancellation rate across all orders

COUNTIF gives 212 of 3,000. **AVG** of the 0/1 column gives the rate.

```
SELECT ROUND(AVG(  
    CASE WHEN status = 'cancelled'  
        THEN 1 ELSE 0 END  
    ), 4) AS cancellation_rate  
FROM nakheel.orders;
```

→ **0.0707**

BETWEEN on a timestamp drops the last day

The same words on a different column type, and five orders go missing.

Suggested time • 5 minutes

[labs/D04/11-Ins-Between-Timestamp/README.md](#)

BETWEEN on order_ts loses 31 August

'2026-08-31' on a timestamp means midnight at the start of the day. 153, not 158.

```
SELECT COUNT(*) AS august_orders  
FROM nakheel.orders
```

```
WHERE order_ts BETWEEN '2026-08-01' AND '2026-08-31';
```

→ 153

Up to, not including, the next month

From the first moment of August to just before September.

```
SELECT COUNT(*) AS august_orders  
FROM nakheel.orders  
WHERE order_ts >= '2026-08-01'  
      AND order_ts < '2026-09-01';
```

→ 158

KPIs, and the lab

Write the rule down before you calculate the number.



A KPI definition table

Name, rule in words, grain. Then one query for all three.

Suggested time • 15 minutes

[labs/D04/12-Evr-KPI-Definitions/README.md](https://github.com/GoogleCloudPlatform/GoogleCloudDataFusion/blob/master/labs/D04/12-Evr-KPI-Definitions/README.md)

DEFINITION

A KPI, key performance indicator, is a number a business agrees to calculate the same way every time.

Three KPIs, one grain

One query returns all three, 24 rows. September 2024: 11,268.00 from 112 orders, average 100.61.

KPI	Rule in words	One row is
Completed revenue	Sum of order_total, completed orders	one month
Completed orders	Number of completed orders	one month
Average order value	Completed revenue ÷ completed orders	one month

The misleading average

What does a typical Nakheel order cost? Take two answers first.

Suggested time • 10 minutes

[labs/D04/13-Ins-Misleading-Average/README.md](#)

What does a typical order cost?

Average line value

45.46

One row is one product on one order

Average order value

114.05

One row is one order

2,082 of the 2,771 completed orders are below 114.05.

Launch the lab

Five KPIs for Nakheel's managers. The definition table comes first.

Suggested time • 5 minutes

[labs/D04/14-Lab-KPI-Set/README.md](#)

Break

 Back at 7:00 PM

The TAs run the lab from 7:00. Ask in your group channel, with a screenshot.

Lab: a KPI set for Nakheel

TA-led. Five KPIs, and one answer that changes with the grain.

Lab: A KPI set for Nakheel Retail

In this lab, you will define and calculate five KPIs, then see how one answer changes with the grain.

Suggested time

45 minutes

[labs/D04/14-Lab-KPI-Set/README.md](#)

Lab: A KPI set for Nakheel Retail

CHECKPOINTS

- C1 Revenue and orders by month; average order value by channel.
- C2 Cancellation rate by month, and the months above 9%.
- C3 Is the typical order getting bigger? Year, month, then like with like.

HINT

For C2, WHERE status = 'cancelled' removes the orders you divide by.

STRETCH

Orders by status on thelook_ecommerce.orders; read the bytes first.

Time's up. Let's review.

KPI set

Solved: `labs/D04/14-Lab-KPI-Set/Solved/kpi_set.sql`

CLOSE

What you can do now

- State the grain before you group
- Count three ways and explain the difference
- Filter groups with HAVING
- Turn a written rule into CASE

NEXT

Save today's .sql file. D05: joins and the fan-out trap.